
Detecting Fraud with Bayesian Networks

Lucas Hlaing

Mia Chen

Dhruv Agarwal

Jude Gamba

Alex Gao

1 Problem Description

The problem we are trying to solve for our project is fraud detection, specifically the detection of bank account fraud. When a bank receives an application for a loan or a credit card, the bank must determine if the application is fraudulent or real. With more and more personal data being requested and used online, this is a growing problem facing many people. Quite often a story about a large data leak, or about cases of identity fraud will spread, further highlighting the harms that come from bank account fraud. In this project, we aim to predict whether or not a given application is fraudulent or not given a variety of different data points/markers.

The problem itself is inherently a probabilistic one, as when an application comes in, there's going to be a certain probability that the application is fraudulent. Banks have to make a decision on whether or not they believe the application is fraudulent. The task is done given a variety of different data points, such as credit score, email, or even the OS the application was sent on. The bank then takes all the data points it has and makes an educated guess as to whether or not it believes the application is fraud or not. This is exactly what an inference question is, and is the exact kind of probabilistic question we've studied in class: $P(\text{Fraud}|\text{Evidence})$. We can train a model that we learned in class, and use some of the learning and inference algorithms for fraud detection.

2 Data Sourcing and Processing

For our dataset, we used "Bank Account Fraud Dataset Suite (NeurIPS 2022)" on Kaggle. The dataset consists of 6 different synthetic datasets designed to be similar to real world situations. For the purposes of our model, we will be utilizing the base version of the dataset, consisting of 31 features and labels for ground truth. The dataset directly corresponds to the the fraud detection problem, as it is designed to be representative of real data. Normal datasets for fraud detection usually involve some kind of anonymization, which usually include removal of feature labels, as well as normalization of values, making it much harder to understand relationships. However, given the data is synthetic, there are no guarantees that the dataset is actually representative of a real life situation, and thus the model might not be applicable to a specific situation.

For pre-processing, we broke the features we were using for a given model into separate discretized buckets. For the larger models, we utilize 10 hand-selected features: "customer_age", "name_email_similarity", "income", "bank_months_count", "payment_type", "keep_alive_session", "has_other_cards", "velocity_24h", "credit_risk_score", and "email_is_free". For our smaller models, we use "customer_age", "income", "bank_months_count", and "credit_risk_score". These features were selected as they seemed like common sense features that could be used to detect fraud.

As our data contained mainly non-frauds with frauds making up only 1% of the dataset, we rebalanced the dataset to have a ratio of 1 to 5, meaning for every fraud, we have 5 non-fraud datapoints. These non-fraud datapoints were randomly selected from the full pool while we used all the fraud datapoints available, due to scarcity.

3 Modeling and Inference

3.1 Model

For the model, we are utilizing a Naive Bayes structure, which assumes our features are conditionally independent given the fraud label. This allows us to perform inference using Bayes Rule. There are obviously limitations to the Naive Bayes assumption, as it can't capture any of the dependencies between features. Despite this, the Naive Bayes model allows for fast training and inference.

We create two Naive Bayes structures, one small (containing features "customer_age", "income", "credit_risk_score", "bank_months_count") and one big (containing all ten features). We will be utilizing two ways in which to learn the CPTs of each structure, Maximum Likelihood Estimation (MLE) and Expectation Maximization (EM), and compare those results.

3.2 Maximum Likelihood Estimation (MLE)

MLE estimates the CPTs that maximize the likelihood of observing the data given in our training dataset. Since we are utilizing the "bank_months_count" feature, which contains missing values, we had to perform some data preprocessing in order to use MLE, which expects complete data. To this end, we used the mean of existing "bank_months_count" values to perform imputation on missing values.

3.3 Expectation Maximization (EM)

Expectation Maximization takes an iterative approach to estimating CPTs and can be utilized when there is incomplete data. In our case, "bank_months_count" is missing in some examples, so we chose to use this as our latent variable. Due to the constraints of the library we used (pgmpy), "bank_months_count" is treated as a hidden variable for all examples. The column is essentially dropped from the dataset for EM. This could be viewed as a limitation of the specific library we used.

4 Results and Discussion

4.1 MLE

Fraud	Precision	Recall
0	.85	.97
1	.54	.17

Table 1: Small BN MLE Performance

Fraud	Precision	Recall
0	.87	.96
1	.57	.28

Table 2: Large BN MLE Performance

4.2 EM

Fraud	Precision	Recall
0	.85	.98
1	.54	.12

Table 3: Small BN EM Performance

4.3 Discussion

As we can see, the precision and recall are quite low when it comes to fraud detection. The increase in recall from the smaller BN to the larger BN means that the increase in data does lead to a higher

Fraud	Precision	Recall
0	.85	.99
1	.57	.10

Table 4: Small BN EM Performance with finer grain buckets

Fraud	Precision	Recall
0	.87	.96
1	.59	.27

Table 5: Large BN EM Performance

fraud detection rate when it comes to cases of fraud. The precision and recall combined tell us that a lot of the time, the evidence just isn't strong enough to overcome the already unlikely nature of fraud.

Given the nature of our model to begin with, the model couldn't capture any relationships between features. It did, however, seem to infer some reasonable assumptions within the CPTs. Take this CPT:

This is the CPT for the "income" feature, where we can see that given the label is fraud, there's a .44 probability that the application is for someone with a high income. This makes intuitive sense, as opening a fraudulent bank account under someone of high income would result in more gain for the fraudster.

It is possible that for MLE, the use of an average is insufficient. Another alternative that could be explored in future work could be utilizing the missing data as data itself. That is, assigning missing data a bucket. The reasoning behind this for a feature like "bank_months_count" is that a missing entry very likely implies that it's a completely new account being opened. Lack of data could also be a reason for the low precision and recall, as we utilized 10 features as opposed to the full 31. Another potential source of error was the discretization of values. Since many of these values were continuous, we had to discretize feature values, with the chosen method being evenly splitting up the range of values for each feature and assigning each entry a bucket based on it's value.

For EM, similar actions such as utilizing the full dataset could increase the performance, but an EM specific change could also be utilizing a different library that supports partially observed latent variables like "bank_months_count" instead of dropping it entirely. We also tried another test where we changed how many buckets there were for certain features like "income". As shown in the figures above, there was a mix of increases and decreases in precision and recall for small and large BNs.

Interestingly enough, the performance of EM and MLE were extremely similar. This is most likely due to how similar the data sets we used to compare both were. Both algorithms were given BNs utilizing identical datasets apart from the "bank_months_count" entry, where in EM it was completely hidden, and in MLE missing entries were estimated using the average.

5 Conclusion

Our project tried to solve the problem of fraud detection, specifically in bank account fraud. Across small and large models, as well as training methods of MLE and EM, there was high performance in predicting non-fraud applications, however there was consistently poor performance in predicting fraudulent applications, which is the more important metric in real-world scenarios.

Even with poor performance, our experiments showed an increase in performance when models were given more data, suggesting that an increase in features used could significantly increase performance. The models also seemed to capture some meaningful relationships in the CPTs as evidenced with the "income" CPT. While finer-grained buckets didn't seem to provide any concrete increases in performance, it still remains to be seen whether or not finer grain buckets in conjunction with other changes could improve performance.

With low precision and recall in both MLE and EM, we found that the Naive Bayes model trained on specific hand-selected features had no appreciable differences training method of MLE and EM. For future work, a different BN structure that can capture relationships between features would most likely greatly increase performance, as well as adding in more features from the dataset as a

Fraud	Precision	Recall
0	.87	.95
1	.57	.30

Table 6: Large BN EM Performance

Fraud	Income Low	Income Med	Income High
0	.36	.42	.22
1	.20	.36	.44

Table 7: Large EM Income CPT

whole. Another change could be the use of continuous models, or finer-grained discretization buckets. Ultimately, while a small Naive Bayes model allowed for fast inference and training, a more complex model most likely would be needed for a high performance predictor.

6 Reflections and Contributions

For similar projects in the future, we would highly recommend taking care of some easy, but important factors such as re-balancing the dataset when training, and utilizing precision and recall when evaluating model performance on highly-imbalanced datasets. When a task is much more focused on predicting a rare event correctly, these 2 metrics better represent actual performance compared to accuracy, and should be used instead. A re-balanced dataset is also crucial as it allows the model to learn relationships between the rare event and the features, otherwise it's entirely possible the model just guesses the more common event.

Lucas: I did the data processing and cleaning portion of the assignment. I've learned how to use pandas dataframes better and the impact that feature selection and data balance can have on results.

Mia: I worked with libraries to perform MLE on the complete data in the small and large networks, practicing imputation with Pandas to replace missing data. This project allowed me to practice using Pandas and also fully understand the applications of EM vs MLE as we were initially deciding on a dataset and methodologies.

Dhruv: I implemented the EM step for the small BN, as well as applying MLE to both small and large networks using the new balanced dataset to address class imbalance and carry out the threshold tuning analysis. I learned how class imbalance can lower recall in fraud detection and how balancing the data improves the models ability to identify fraud cases. I also learned how adjusting the decision threshold changes the tradeoff between predictions and recall and why a lower threshold can be more appropriate for identifying fraud.

Jude: I contributed to creating and running the em step for likelihood estimation for the large network. I've learned the importance of a balanced data set when tuning a model for accurate predictions and how biased data can lead to models that are biased for certain classification tasks. Additionally, tuning model thresholds to encapsulate data can lead to more accurate predictions, but can also lead to tradeoffs with model performance.

Alex: I contributed to reviewing and bringing and making sure everyone was on the same page across EM and MLE, as well as writing a majority of the report. I learned that re-balancing a data set when it comes to predicting rare events is extremely crucial, as well as how to plan and communicate with others to make sure our models and data is consistent across the entire project.